

From Version Conflicts to Decision Conflicts: Selective Revalidation for Long-Running AI Agents

Yongjian Lyu^{1†}

Yang Ren^{1†}

Ruofei Lai¹

¹ Huawei Technologies
Co., Ltd.
United Kingdom

Wenting Liu²

² Xi'an Jiaotong University
Xi'an 710049, China

Abstract

Long-running AI agents may read state, reason, wait for tools or human approval, and perform an external action much later. The state that justified the action can change in the meantime. For example, after an agent proposes an 80 GBP refund under a limit of 100, a customer-name change affects only presentation metadata, a new limit of 90 still permits the refund, a limit of 50 invalidates it, and a refund issued by another worker must prevent a duplicate. Standard optimistic concurrency control and version checks can detect that previously read state has changed, but by themselves do not determine whether that change invalidates the pending action's justification. We call any detected version change a *version conflict*; when that change invalidates the action's justification, it is also a *decision conflict*. ATR records the explicit, executable conditions that justify a pending action and rechecks only the conditions affected by a change before releasing the external operation. It can retain the action, refresh non-decisive metadata, require replanning, or block execution; a target-side transaction or compare-and-set binds checked state to commit. Across 210,000 controlled executions over 15 mutation cases, ATR matched every developer-specified outcome with no false allows or blocks. In ten durable SQLite checkpoint/resume cells, it evaluated 0.6 conditions per change versus 6.0 for FullScan. At 4,093 recorded reads, ATR took 9.3 μ s versus 2595.9 μ s for FullScan. These deterministic results establish controlled feasibility, not production generality or automatic extraction of the required conditions.

Keywords

AI agents, transaction management, decision consistency, semantic validation, provenance

1 Introduction

AI agents interleave reasoning, retrieval, and tool actions [15]; durable executions may also cross approval pauses, retries, and resumptions [12]. This separates the time at which an agent reads state from the time at which it acts on that state. Consider a refund agent that observes an order as delivered, defective, not yet refunded, and worth 80 GBP, together with a delegated refund limit of 100. It concludes that the customer is eligible, proposes the refund, and waits for approval. Before execution, the customer display name may change; the limit may fall to 90 or 50; or another worker may issue the refund. Every update can change a recorded version. The name change and a new limit of 90 preserve the justification; a limit of 50 and a duplicate refund invalidate it.

[†]Corresponding authors: Yongjian Lyu (lyongjian@huawei.com) and Yang Ren (renyang1@huawei.com).

Database transactions and optimistic concurrency control (OCC) remain responsible for target-local concurrency and atomic updates [2, 8]. They can report that an agent read version 11 while the resource is now at version 12. They do not know whether the changed field matters to the refund. Semantic serializability can use application meaning to admit more database schedules, but it remains a correctness criterion for transaction execution [3]. An agent may instead reason across databases, APIs, documents, and long pauses, so holding one transaction open is neither practical nor sufficient. The layers answer different questions: OCC asks whether state changed; ATR asks whether the change invalidates the pending action's justification; the target transaction or compare-and-set (CAS) asks whether the action can atomically commit against the checked state.

The key observation is that a version change should trigger inspection, not automatic invalidation of the agent's decision. ATR records the concrete facts used by a decision and the executable conditions that must remain true for its pending action. When related state changes, or immediately before execution, ATR follows the recorded dependencies and rechecks only the affected conditions. Figure 1 shows this boundary: ATR decides whether the proposed action remains justified, while the target database or API still enforces the atomic operation. This separation avoids claims of global serializability or exactly-once execution across unrelated systems.

This short paper contributes: (1) a model that distinguishes version conflicts from decision conflicts; (2) a reverse-index protocol with four operational outcomes and target-side checked-state binding; and (3) an executable prototype and controlled evaluation of safety, selectivity, scaling, races, and dependency omission. Our results support feasibility, not production generality: the workloads use explicit developer-authored conditions and deterministic ground truth.

2 Model and Semantics

2.1 Agent Transactions and Premises

We first ground the terminology in the refund example. *Evidence* is the concrete state observed by the agent: delivered = true, defect = true, refunded = false, amount = 80, and refund-limit = 100. A *decision* is the conclusion supported by those observations—here, that the order is eligible for an 80 GBP refund. A *premise* is a condition that must remain true for the decision or action to stay justified. The refund depends on five premises: the order was delivered; a defect exists; no refund has been issued; 80 does not exceed the current limit; and the agent has refund authority. A *typed premise* represents such a condition with defined fields and semantics so an executable validator can check it again. Free-form chain-of-thought or rationale may aid an audit, but is not itself a premise on which ATR relies. The *intent* is the proposed

Agent execution path

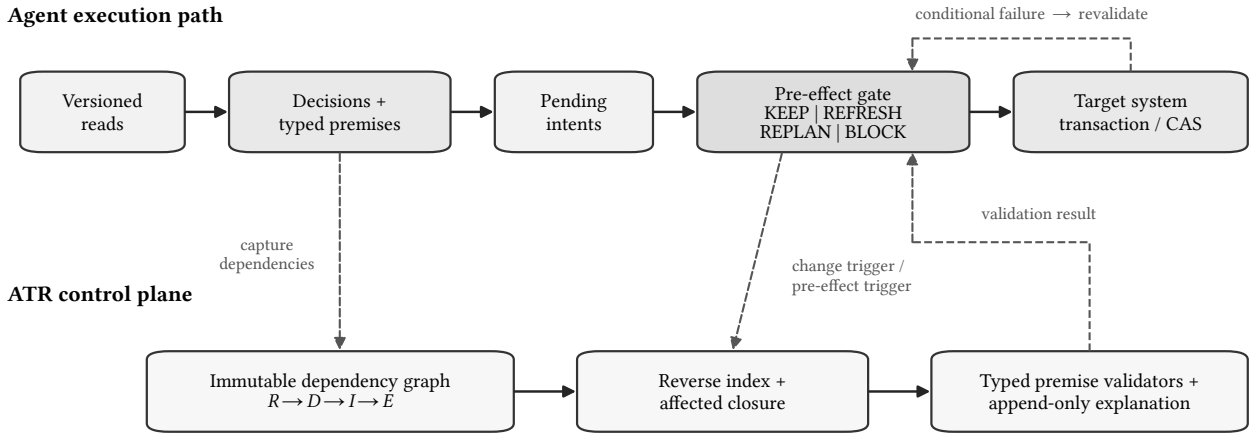


Figure 1: ATR runtime overview. From a changed read, ATR follows recorded dependencies to the reachable pending actions (the affected closure) and rechecks their required conditions. Permitted operations execute through a target-side transaction or CAS; a conditional failure triggers revalidation. State held in another system requires a separate binding mechanism.

“issue an 80 GBP refund” operation; the *effect* is the actual external operation that commits or sends the refund.

Formally, an agent transaction is $T = \langle R, D, I, E \rangle$. In the example, R contains the order and policy observations, D the eligibility decision, I the proposed refund, and E the attempted or committed refund operation. Each read $r \in R$ stores a resource key, value digest, connector-defined version, and read policy. Each decision $d \in D$ stores its outcome and typed premises. A directed graph $G = (V, \mathcal{A})$ contains these immutable nodes, with $u \rightarrow v$ meaning that v may depend on u . The common path is $R \rightarrow D \rightarrow I \rightarrow E$, with many-to-many edges. For a changed read, the *affected closure* is the downstream part of this graph that can reach a pending action; a reverse index locates its starting nodes.

A premise p records evidence fields X , a deterministic validator f , declared failure severity $sev(p)$, and validator/policy version q . Here $sev(p)$ is REPLAN or BLOCK. Provenance standards provide a vocabulary for entities and derivations [11]; ATR adds explicit condition conditions and the gate before an external operation.

Let $P(e)$ contain all required premises reachable from pending effect e ; a premise is required when its failure changes the operational response for the current intent. A premise is *block-critical* when its failure severity is BLOCK, or policy marks it as safety-, authority-, compliance-, or irreversible-effect-critical. Let $B(e)$, the *binding set*, be the evidence versions or tokens that the target can check atomically when applying e . Given current state S_v at versions v , the effect is *decision-consistent at the gate* when every required $p \in P(e)$ evaluates true, except that an explicit refresh-only rule may update non-decisive metadata. This is a point-in-time property. It becomes a commit-time safety property only for required evidence covered by $B(e)$; it does not assert that the agent chose the optimal action.

2.2 Version and Decision Conflicts

A *version conflict* exists when a resource version differs from the one recorded in R . A *decision conflict* exists when current evidence falsifies or makes unverifiable a reachable premise required by a

Table 1: Validation outcomes. Severity increases from top to bottom; dominance is BLOCK > REPLAN > REFRESH > KEEP.

Outcome	Required runtime response
KEEP	Retain the current intent; no recorded premise is falsified and no refresh-only rule fires.
REFRESH	An explicit rule refreshes non-decisive presentation or derived metadata; no decision premise is false.
REPLAN	The current intent is no longer justified, although the task may remain feasible.
BLOCK	Do not release the effect; a safety, authority, policy, or irreversible-effect condition is false, missing, contradictory, or unverifiable.

pending intent. The refund cases in Table 2 make the distinction explicit. All five changes advance a recorded version. A new limit of 90 preserves $80 \leq 90$ and therefore returns KEEP; a limit of 50 falsifies the delegated-limit premise, and an already-issued refund falsifies the not-yet-refunded premise, so both return BLOCK. A display-name change affects only presentation and invokes an explicit REFRESH rule. A corrected defect flag is a typical REPLAN case: the current refund is no longer justified, but the task may be reconsidered. A version conflict is therefore a trigger for inspection, not by itself a reason to discard the plan. Table 1 defines these responses.

2.3 Conditional Safety and Localization

ATR maintains four invariants: every gated effect references its intent and supporting decisions; every decision has typed premises or conservative edges from every supplied input; every validation record names the evidence and validator versions checked; and explanations are append-only. These invariants support the following conditional property.

Proposition 1 (bound pre-effect safety). Assume (i) dependency completeness: every evidence item capable of affecting e

Table 2: Post-decision changes for the proposed 80 GBP refund. Every row changes a connector-local version.

Change after decision	Required condition	Outcome
Display name changes	No required premise changes	REFRESH
Limit: 100 $\rightarrow$ 90	$80 \leq 90$ remains true	KEEP
Defect: true $\rightarrow$ false	Defect premise becomes false	REPLAN
Limit: 100 $\rightarrow$ 50	Delegated-limit premise is false	BLOCK
Another worker refunds	Not-yet-refunded premise is false	BLOCK

has a recorded path to e ; (ii) each validator is sound for its declared typed premise; (iii) token faithfulness: any change capable of altering a bound premise changes its connector version or token; (iv) observation binding: the token supplied to the target is the exact token paired with the observation evaluated by the validator; and (v) atomic target enforcement: the target accepts e only while every required bound token in $B(e)$ still matches. Then an effect with a false block-critical premise covered by $B(e)$ cannot commit through the ATR gate.

Proofsketch. Dependency completeness places the changed evidence and supporting decision in the affected closure. Validator soundness maps a false block-critical premise to BLOCK, which dominates the other outcomes, so the gate does not invoke the effect. If a bound premise changes after validation, token faithfulness and observation binding make the atomic target condition fail; the rejected apply is not a commit and returns to validation. A target-only CAS does not protect an external policy, budget, or authorization resource whose token is absent from $B(e)$. The proposition therefore does not claim global serializability, cross-system atomicity, exactly-once effects, or complete recovery, and it does not cover an omitted premise or unsound validator.

Proposition 2 (graph-relative localization). With exact reverse-index lookups, affected closure computation visits only nodes reachable from matching reads, in time $O(|\mathcal{A}_c| + |V_c| + L)$ for lookup cost L and visited subgraph $(V_c, \mathcal{A}_c)$. The traversal is confined to the subgraph reachable under the recorded dependency relation; conservative edges may include irrelevant nodes, and missing edges are unsafe.

3 Runtime Design

3.1 Capture and Change Localization

Connectors return resource identity, current value, version descriptor, and conditional-effect capability. Versions may be ordered counters, commit timestamps, ETags, hashes, or equality-only tokens. ATR records the actual observation; monotonic or compatible caching can improve the read path but does not determine whether a pending decision survives a later change. MCCache, for example, enforces monotonic and consistent cache observations for transaction-like requests [1]; ATR instead links observations to decisions and effects.

Dependencies can be captured conservatively for every input supplied to a reasoning step, explicitly by application code, or from structured model output. Reverse-index entries may select a whole decision or premise identifiers attached to it. Object/field capture revisits every premise on a selected decision; predicate capture can revisit only explicitly indexed premises. Thus predicate capture may run one validator where coarser capture runs all validators on that decision.

Algorithm 1 Affected closure validation. Status ordering is BLOCK > REPLAN > REFRESH > KEEP.

```

1: procedure Validate( $x, e$ )
2:  $\Delta \leftarrow \text{normalize\_event}(x)$ ;  $obs \leftarrow \emptyset$ 
3: if  $x = \text{PRE\_EFFECT}$  then
4:    $obs \leftarrow \text{read\_current}(\text{required\_evidence}(e))$ 
5:    $\Delta \leftarrow \Delta \cup \text{version\_diff}(e, obs)$ 
6: end if
7:  $C_{\text{aff}} \leftarrow \text{reachable\_pending}(\text{index}(\Delta))$ 
8: for all  $p \in \text{required\_premises}(C_{\text{aff}})$  do
9:   if  $X(p) \not\subseteq \text{dom}(obs)$  then
10:     $obs \leftarrow obs \cup \text{read\_current}(X(p) \setminus \text{dom}(obs))$ 
11:   end if
12:    $\sigma[p] \leftarrow \text{validate}(p, \text{observations\_for}(X(p), obs))$ 
13: end for
14:  $o \leftarrow \text{highest\_status}(\sigma, \text{refresh\_rules}(\Delta))$ 
15:  $(o_b, b) \leftarrow \text{binding\_policy}(e, obs, B(e))$ ;  $o \leftarrow \max(o, o_b)$ 
16:  $a \leftarrow \text{NOT\_ATTEMPTED}$ 
17: if  $x = \text{PRE\_EFFECT} \wedge \text{permits}(o, e)$  then
18:    $a \leftarrow \text{conditional\_apply}(e, b)$ 
19:   if  $a = \text{REJECTED}$  then
20:      $\text{schedule\_revalidation}(e)$ 
21:   end if
22: end if
23:  $\text{append\_explanation}(x, C_{\text{aff}}, \sigma, o, b, a)$ 
24: return  $(o, a)$ 

```

Free-form rationale cannot authorize an effect. When required typed premises are known but dependency attribution is uncertain, ATR may link every supplied input conservatively. This avoids missing-edge false allows but cannot repair an omitted or unsound premise. A changed conservatively linked input without a precise validator maps to REPLAN or BLOCK; authority- or irreversible-effect-critical cases fail closed.

Change streams and webhooks are early-warning optimizations, not the correctness path. NORMALIZE_EVENT converts a hint into changed resource/field pairs Δ ; VERSION_DIFF compares every required evidence token at PRE_EFFECT, including REPLAN premises. Each connector returns a value and token as one observation, so b contains the exact tokens paired with the values validated. REFRESH_RULES contains explicit application rules for non-decisive metadata. BINDING_POLICY returns BLOCK for unbound authority, safety, or irreversible-effect evidence; any lower-risk relaxation must be explicit. Object-only mismatches conservatively seed the resource. A missing or timed-out validator follows the same REPLAN/BLOCK policy. Apply status a remains separate from the four validation outcomes; rejection never means that a KEEP effect committed.

3.2 Closing the Validation–Effect Race

Validation alone leaves a time-of-check/time-of-use gap. For target-local premises, ATR passes the checked target version into a native transaction, CAS, uniqueness constraint, lease, or domain command. If the condition fails, ATR revalidates or blocks. This does not by itself close a race on an external policy, budget, or authorization record: such a change need not advance the target version. Strong binding requires the target transaction to check those versions, or an equivalent reservation, capability, lease, or fencing token. Otherwise ATR records the evidence as unbound and exposes the residual race to policy; an irreversible effect should fail closed. Concurrent agents need not serialize their full reasoning, but the guarantee is limited to the tokens the authoritative target actually checks.

ATR is not a workflow or recovery protocol. Sagas address long-running recovery through compensating transactions [7]; effect-staging systems can suppress or compensate tool calls. ATR can supply them with decision provenance but does not solve

crashes between an external effect and its acknowledgement, partial multi-system effects, or non-idempotent APIs.

4 Prototype and Methodology

We implemented a compact ATR runtime core in Python 3.11. The prototype uses immutable read/decision/intent records, SHA-256 value digests, hash-based reverse indexes, typed Python validators, append-only JSON records, and an in-memory target exposing a version-token unit check. SQLite harnesses exercise two-client target races, durable checkpoint/resume, and cross-resource commit conditions. The runtime supports object, field, and predicate granularity. The serialized graph averaged 7.4 KiB per trace with 48 extra context reads; this is an uncompressed prototype representation, not a storage lower bound.

The evaluation asks: **RQ1**, does ATR reproduce the declared semantic outcome and prevent invalid effects? **RQ2**, what selectivity and runtime cost follow from reverse traversal? **RQ3**, where do races, unavailable validators, and incomplete dependencies break the guarantee?

Workloads. We built three deterministic, executable scenarios: refund (delivery, defect, duplicate-refund, and delegated-limit premises), cloud remediation (incident state, duplicate remediation, restart authority, and maintenance freeze), and procurement (request state, approved vendor, budget, auto-approval cap, and purchasing authority). Each scenario has five mutations: one irrelevant (KEEP), one presentation-only (REFRESH), one decision-invalidating (REPLAN), and two authority/target-critical (BLOCK). Thus, 40% of the fixed mutation mix is safe for the existing intent and 60% is not. The oracle is the same explicit business rule used to define each mutation category; no LLM labels results.

Durable workflow cells. Following CoAgent’s evaluation pattern of adding contending peers to existing tasks [9], we construct one office-automation pattern and one cloud-remediation pattern, inspired by task patterns represented in WorkBench [13] and AIOpsLab [5]. For each, five intervening changes from a second actor comprise an irrelevant update, a presentation-only update, a change that preserves a predicate, a replanning case, and a blocking case. The workflow checkpoints to SQLite, closes its connection, a second connection changes shared state, and a fresh connection rebuilds the validation graph from checkpoint state and the versioned validator registry before gating the effect. Ten trials per cell and method yield 400 method executions (10 cells $\times$ 10 repetitions $\times$ 4 methods). These deterministic patterns do not run an LLM and are not full upstream benchmark suites; they test persistence and target interaction beyond the in-memory core.

Baselines and metrics. *NoCheck* releases the original intent. *VersionAbort* replans on every version mismatch. *FullScan* scans all recorded provenance without a reverse index and re-evaluates every typed premise; it does not replay the LLM or workflow. We report invalid effects, false allows/blocks against the oracle, unnecessary replans, closure size, serialized bytes, and in-process validation latency. Main and scaling latency measure the warmed in-process validation call, including traversal and typed premise validators, but excluding graph construction, serialization, logging, checkpoint I/O, connector/network calls, model calls, and replanning. The separate durable-cell timer includes SQLite connect/read, JSON restoration, graph reconstruction, validation, and the conditional update; those timings are not plotted or used in Table 3.

Table 3: Summary metrics. Unsafe-release and safe-replan rates are conditioned on their respective controlled mutation classes; premises/change is measured on durable workflow cells; p50 latency measures in-process validation.

Method	Unsafe released (%)	Safe replanned (%)	Premises / change	p50 (μ s)
NoCheck	100.0	0.0	—	—
VersionAbort	0.0	100.0	—	—
FullScan	0.0	0.0	6.0	39.9
ATR	0.0	0.0	0.6	10.8

We run 7 repeated balanced schedules with distinct shuffle seeds, 10,000 traces per workload per repetition (210,000 executions), on 16 vCPUs of an Intel Xeon Gold 6266C with 62 GiB RAM and Linux 6.6. For each repetition we checkpoint raw trace rows. Scaling holds the affected predicate constant while increasing unrelated recorded reads. We also run 7,000 two-client SQLite conditional-update races across seven repetitions, fail-closed validator tests, and a dependency-omission fault-injection study with 35,000 unsafe trials at each omission rate. Source, commands, checkpoints, raw CSV, independent audit scripts, and summary JSON are in the artifact.

For the binding boundary, we repeat one scheduled history 100 times per mode: validate, change an external refund policy, then attempt the target effect. The repetitions stabilize timing only; they are not distinct histories. The target-only mode checks the order version, while the bound mode atomically checks both the order and recorded policy token.

The 210,000 executions repeat 15 semantic mutation cases (three workloads times five mutations); they are not 210,000 distinct scenarios. The repetitions check deterministic outcome agreement and measure runtime variation under shuffled schedules.

5 Results

Controlled safety and selectivity. Table 3 reports the core result. *NoCheck* released 126,000 invalid effects (60.0%), a direct consequence of the controlled mutation mix. *VersionAbort* released none but replanned all 84,000 safe changes. ATR produced the oracle outcome for every trace (0 false allows and 0 false blocks), retaining or refreshing those 84,000 intents. These observations demonstrate correct implementation for the declared rules; they do not estimate real-world change frequencies or premise-extraction accuracy.

Durable workflow cells. Figure 2(a) reports exact outcomes for the local durable integrations. Across ten semantic cells, *NoCheck* and *VersionAbort* matched the exact operational outcome in 40% and 20%, respectively; *FullScan* and ATR matched all 100 executions per method. As summarized in Table 3, ATR evaluated 0.6 typed premises per resumed change, versus 6.0 for *FullScan*. These results establish durable checkpoint/resume and SQLite interaction for the declared cells, not agent reasoning quality.

Cost and granularity. ATR’s p50/p95/p99 in-process validation latency is 10.8/13.0/20.5 μ s; per-repetition p50 is 10.77 $\pm$ 0.07 μ s (95% CI). Predicate traversal visits 2.6 nodes on average (p95 3), versus 6.4 (p95 8) for whole-object capture. *FullScan* is 39.9 μ s at the workload trace size. When unrelated reads increase from 13 to 4,093, ATR changes from 8.8 to 9.3 μ s, whereas *FullScan* rises from 12.1 to 2595.9 μ s (Figure 2(b)). This is a single-process

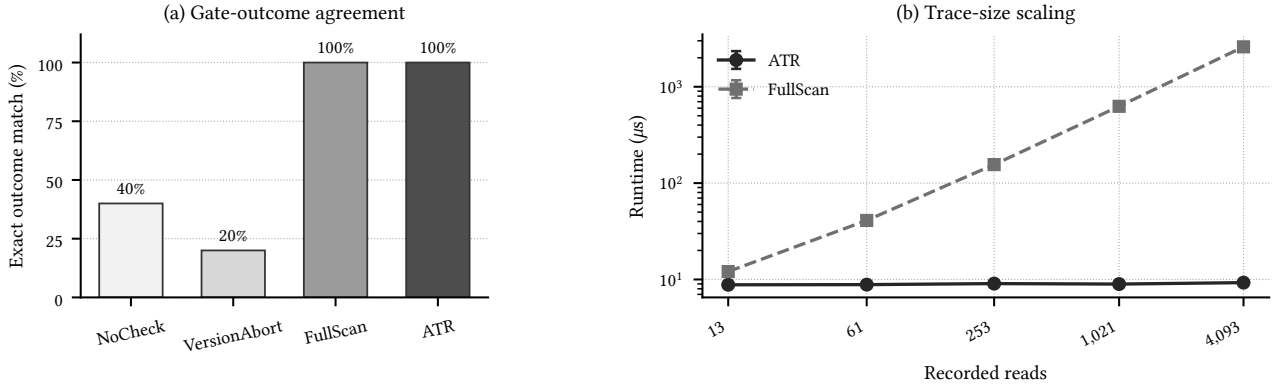


Figure 2: Gate-outcome agreement and trace-size scaling. (a) Exact four-way outcome agreement on ten controlled durable workflow cells. (b) In-process ATR and FullScan runtime versus recorded reads (log-log axes; 95% CIs over seven repetitions).

microbenchmark; distributed-store and connector latency remain to be measured.

Races and failure modes. Across 7,000 pairs, two independent SQLite connections were released at a barrier to issue the same atomic version-conditional update. Exactly one update committed per pair (0 double-commit and 0 zero-commit pairs). This local-backend test validates the binding pattern, not distributed-database behavior. In the deliberately scheduled cross-resource history, target-only CAS committed 100/100 effects after the policy limit became false; adding the checked policy token rejected 100/100. This is a counterexample and mechanism check, not an estimated production failure rate. Three targeted unavailable-validator tests all returned BLOCK. Randomly omitting the sole critical dependency edge produced false-allow rates of 0.95%, 5.05%, and 9.97% at 1%, 5%, and 10% omission. Selective validation is safe only when dependency capture is complete or conservatively over-approximated.

6 Related Work

The related systems address different layers of a long-running agent action. Database isolation and OCC control read/write conflicts and atomic target updates [2, 8]; semantic serializability uses application semantics to enlarge the set of correct database schedules [3]. ATCC adapts optimistic and pessimistic control to long, unforeseen SQL transactions [16]. These mechanisms govern data execution; they do not determine whether a changed business fact invalidates the reasoning behind a pending agent action.

At the agent-concurrency layer, CoAgent orders concurrent peers, reports conflicts to LLM workers, and repairs or reverses writes to target serializability [9]. ATR neither orders agents nor repairs visible writes; it checks the stated conditions for one pending action. SemIso instead keeps prompts, models, indexes, policies, and tools compatible across durable continuations [12]. It protects the workflow’s semantic environment, while ATR asks whether mutable business evidence still supports a particular decision.

Other work addresses admission, effect release, or recovery. Mnemosyne admits generated proposals under executable constraints and repairs disruptions [4]. Cordon validates a composed flow before releasing staged effects [6]; Atomix uses progress frontiers to decide when speculative or contending effects may

settle [10]. Sagas provide compensation for long-running recovery [7], and Agentic Transaction proposes a broader semantic-ACID architecture [14]. ATR occupies the narrower decision-validation layer between observation and effect: it asks whether changed evidence has falsified a condition that justified the pending action, then relies on the target system for atomic commit.

7 Limitations and Conclusion

The evidence is deliberately narrow. ATR runs on one host; the task patterns inspired by WorkBench/AIOpsLab are not complete upstream runs; workloads are deterministic; premise validators and ground truth are developer-authored; and no experiment measures natural-language premise extraction, model replanning quality, distributed failure, or production change distributions. A model-generated premise must be treated as untrusted until mapped to an executable validator. When dependency attribution is uncertain but the required typed premises are known, conservatively linking all supplied inputs avoids missing-edge false allows at the cost of additional revalidation. This does not compensate for an omitted or unsound premise.

Within those boundaries, ATR demonstrates that version mismatches need not force an all-or-nothing choice between ignoring change and discarding all prior reasoning. Recording evidence-to-effect dependencies, revisiting the affected closure, and binding the result to checked target tokens can retain harmless decisions while blocking invalid ones at microsecond-scale in-process validation cost. For cross-resource premises, that guarantee requires a transaction or explicit reservation/lease mechanism covering the external evidence. The next step is integration with a durable agent framework and transactional connectors. It should then be evaluated on production-derived traces with measured premise-extraction error.

Generative-AI Use Disclosure

OpenAI Codex was used for prototype and code scaffolding, experiment-script review, figure generation, and language editing. Executable rules and evaluation labels are developer-authored, and reported aggregates were independently recomputed from raw checkpoints. The human authors inspected and take responsibility for all submitted code, data, citations, and claims.

Artifacts

The public artifact is available in the GitHub repository `ezreal13/atr-decision-validation` (commit `89c32fa`). It contains the complete Python prototype, workload and mutation definitions, tests, controlled and anchored checkpoints, raw CSV, SQLite databases and outputs, binding traces, independent audit reports and scripts, summary JSON, the figure generator and editable figure sources, and exact reproduction commands. The experiment requires Python 3.11 and NumPy; it runs without network access or model credentials. Each repetition can be resumed independently.

References

- [1] Shuai An and Yang Cao. 2022. Making Cache Monotonic and Consistent. *Proceedings of the VLDB Endowment* 16, 4 (2022), 891–904. doi:10.14778/3574245.3574271
- [2] Hal Berenson, Philip A. Bernstein, Jim Gray, Jim Melton, Elizabeth J. O’Neil, and Patrick E. O’Neil. 1995. A Critique of ANSI SQL Isolation Levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. 1–10. doi:10.1145/223784.223785
- [3] Angelo Brayner, Theo Härder, and Norbert Ritter. 1999. Semantic Serializability: A Correctness Criterion for Processing Transactions in Advanced Database Applications. *Data & Knowledge Engineering* 31, 1 (1999), 1–24. doi:10.1016/S0169-023X(99)00014-2
- [4] Edward Y. Chang, Longling Geng, and Emily J. Chang. 2026. Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows. doi:10.48550/arXiv:2607.00269 arXiv:2607.00269 [cs.AI]
- [5] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *Proceedings of Machine Learning and Systems*, Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/d1f9e4a9f109b6e8b75ed362736f22ec-Paper-Conference.pdf
- [6] Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, and Jidong Zhai. 2026. Cordon: Semantic Transactions for Tool-Using LLM Agents. doi:10.48550/arXiv:2606.17573 arXiv:2606.17573 [cs.OS]
- [7] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*. 249–259. doi:10.1145/38713.38742
- [8] H. T. Kung and John T. Robinson. 1981. On Optimistic Methods for Concurrency Control. *ACM Transactions on Database Systems* 6, 2 (1981), 213–226. doi:10.1145/319566.319567
- [9] Hongtao Lyu, Dingyan Zhang, Mingyu Wu, Xingda Wei, and Haibo Chen. 2026. CoAgent: Concurrency Control for Multi-Agent Systems. doi:10.48550/arXiv:2606.15376 arXiv:2606.15376 [cs.DC]
- [10] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. 2026. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. In *ICLR 2026 Workshop on Agents in the Wild*. OpenReview.net. <https://openreview.net/forum?id=UeRbEpSVUz>
- [11] Luc Moreau and Paolo Missier (Eds.). 2013. *PROV-DM: The PROV Data Model*. World Wide Web Consortium. <https://www.w3.org/TR/prov-dm/> W3C Recommendation, 30 April 2013.
- [12] Barzan Mozafari. 2026. BEGIN AI TRANSACTION: Semantic Isolation for Durable AI Workflows. doi:10.48550/arXiv:2608.05412 arXiv:2608.05412 [cs.DB]
- [13] Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. 2024. WorkBench: a Benchmark Dataset for Agents in a Realistic Workplace Setting. In *First Conference on Language Modeling*. OpenReview.net. <https://openreview.net/forum?id=4HNawZFDcH>
- [14] Zhaoyan Sun, Xiaoxiao Wang, and Guoliang Li. 2026. Agentic Transaction: Towards ACID-Compliant Agent Systems. doi:10.48550/arXiv:2608.13900 arXiv:2608.13900 [cs.DB]
- [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X
- [16] Weixing Zhou, Zhiyou Wang, Zeshun Peng, Hetian Chen, Yanfeng Zhang, and Ge Yu. 2026. ATCC: Adaptive Concurrency Control for Unforeseen Agentic Transactions. doi:10.48550/arXiv:2603.13906 arXiv:2603.13906 [cs.DB]